

1.

a. Angular devuelve un Observable cuando se realiza una petición HTTP porque estas son asincrónicas, la respuesta no es inmediata, por lo que angular retorna un Observable que permite seguir la ejecución mientras que el servidor envía la respuesta. La diferencia principal que se encuentra al comparar una Promise con un Observable es que el observable permite emitir múltiples valores a lo largo del tiempo, mientras que la Promise solo puede resolver un valor a la vez.

b. Si un componente llama a un servicio HTTP sin hacer `.subscribe()` no se ejecutará la petición y angular solo creará el Observable pero nunca enviará la solicitud.

2.

c. `providedIn: 'root'` permite que Angular maneje automáticamente una única instancia compartida del servicio para toda la aplicación. Esto va a mejorar la reutilización, reduce el consumo de memoria y desacopla los componentes de la creación manual de los componentes. Si se instanciara manualmente con `new CityService()`, el componente quedaría fuertemente acoplado al servicio y Angular no podría inyectar automáticamente otras dependencias internas del servicio.

d. La inyección de dependencias facilita la creación de pruebas unitarias para estos servicios porque permite crear versiones simuladas de los servicios que retorne datos controlados lo que permite comprobar componentes sin depender de APIs externas ni conexiones estables.

3.

e. Centralizar el manejo de errores en `HttpInterceptor` nos evita reutilizar lógica en diferentes componentes y mantiene el código más limpio y mantenible. Además como todas las peticiones `Http` pasan por el interceptor, aseguramos un comportamiento consistente para todos los errores.

f.

1. Agregar automáticamente tokens de autenticación (JWT) en los headers de cada petición HTTP.

2. Registrar logs o métricas de todas las solicitudes y respuestas realizadas por la aplicación.

4.

g. Usar @Input evita peticiones Http innecesarias, ya que la información de la ciudad ya fue obtenida previamente de la lista principal. Esto mejora el rendimiento, reduce tráfico de red y hace que la interacción entre componentes sea más eficiente.

h. Esto simplifica el trabajo del frontend porque toda la información necesaria ya llega en una sola respuesta. Así, el frontend no necesita hacer peticiones adicionales para obtener los datos del país asociado, reduciendo complejidad, tiempo de carga y cantidad de solicitudes HTTP.